

---

# UNIVERSAL MINER PRO

## ULTIMATE EDITION v4.0

---

### COMPLETE TECHNICAL DOCUMENTATION

[ Multi-Pool ] [ Multi-Coin ] [ Multi-Arch ] [ Dual-Algo ]  
[ Swarm Mining ] [ HD Wallet ] [ Profit Switcher ] [ Hardware AI ]  
[ Flask Dashboard ] [ REST API ] [ Gunicorn ] [ SQLite ]  
[ Android/Termux ] [ Linux ] [ macOS ] [ Windows ] [ Raspberry Pi ]

---

A self-contained, autonomous cryptocurrency mining management platform capable of operating 41 simultaneous miner processes across any CPU/GPU/ASIC hardware, with real blockchain wallet integration, live profit switching, and a browser-based control dashboard.

Compiled: June 2026 | Package: UMP\_ULTIMATE\_v4\_0\_COMPLETE  
Architecture: armv7l / aarch64 / x86\_64 / macOS / Windows

---

# TABLE OF CONTENTS

---

1. System Overview & Purpose
2. Architecture & File Structure
3. Core Modules - Detailed Breakdown
4. Supported Hardware & Platforms
5. Coin Registry - All Supported Cryptocurrencies
6. Mining Pool Integration
7. Swarm Engine - 41-Node Parallel Mining
8. Dual-Algorithm System (RandomX / X11)
9. HD Wallet System - Full Blockchain Wallet
10. Profit Switching Daemon
11. Hardware Detection Engine
12. REST API - All Endpoints
13. Database Schema & Storage
14. Dashboard UI & Frontend
15. Installation & Deployment
16. Configuration Reference
17. Security Architecture
18. Roadmap & Future Capabilities
19. Technical Appendix

# 1. SYSTEM OVERVIEW & PURPOSE

Universal Miner Pro (UMP) ULTIMATE Edition v4.0 is a fully self-contained, production-grade cryptocurrency mining management platform. It is designed to run on any hardware from a low-end Android phone to a multi-GPU Linux server, providing a unified browser-based dashboard, a complete REST API, autonomous profit switching, a real self-custodial HD blockchain wallet, and a 41-node parallel mining swarm engine.

The system was engineered with a hard constraint: zero dependency on external hosted services for core functionality. All profiles, session history, wallet keys, and mining statistics are stored locally in a SQLite database. The only external network calls are to mining pool stratums, price oracles (CoinGecko), and blockchain explorers for live balance queries.

## What It Is

UMP is a Python/Flask web application that wraps XMRig and cpuminer-multi mining binaries with a full management layer. It auto-detects the host platform and CPU architecture at startup, builds or locates the appropriate mining binary, and presents a clean terminal-styled green-on-black browser interface accessible from any device on the same network.

## Core Capabilities

| CAPABILITY                 | DESCRIPTION                                                                                                                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autonomous Swarm Mining    | Spawn and manage 40 swarm nodes plus 1 master miner process simultaneously, each as a real operating system subprocess with its own log stream and hashrate parser.                                                    |
| Multi-Pool Multi-Coin      | Support for 7 named mining pools covering 30+ coins across CPU, GPU, and ASIC hardware categories.                                                                                                                     |
| Dual Algorithm             | Switch live between RandomX (XMRig) and X11 (cpuminer-multi) without restarting the dashboard.                                                                                                                         |
| Real HD Wallet             | BIP39/BIP32/BIP44 hierarchical deterministic wallet with real address generation, transaction signing, and blockchain broadcast for BTC, LTC, DASH, ETH, and XMR.                                                      |
| Profit Auto-Switching      | APScheduler background daemon polls CoinGecko and WhatToMine every 5 minutes, scores all minable coins against detected hardware, and automatically switches if a better opportunity exceeds a configurable threshold. |
| Hardware Intelligence      | Detects CPU model, core count, architecture, GPU presence (NVIDIA/AMD via nvidia-smi/rocm-smi), available RAM, and builds a mining capability profile at startup.                                                      |
| Session & History Tracking | Every mining session is logged to SQLite with timestamps, hashrate stats, shares accepted/rejected, and daily earnings estimates.                                                                                      |
| Live SSE Stream            | Server-Sent Events endpoint at /stream pushes real-time stats to the dashboard without polling.                                                                                                                        |
| Production WSGI            | Ships with a Gunicorn config tuned for ARM (gthread worker, SSE-compatible timeout=0).                                                                                                                                 |

---

| CAPABILITY                   | DESCRIPTION                                                                                                         |
|------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Binance Exchange Integration | Optional Binance API key support for deposit address queries and withdrawal initiation directly from the wallet UI. |

## 2. ARCHITECTURE & FILE STRUCTURE

UMP follows a modular Flask blueprint architecture. Each major subsystem is an independent Python module that registers its routes with the core Flask app at startup. All modules share a single SQLite database (miner.db) but use separate table namespaces to avoid conflicts.

### Directory Layout

| FILE                                    | PURPOSE                                                                                                                            |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>universal_miner.py</code>         | Flask application entry point. Wires all modules, registers blueprints, defines core routes.                                       |
| <code>miner_config.py</code>            | Central configuration: coin registry, pool registry, algorithm definitions, DB init, profile CRUD, command builder, system info.   |
| <code>miner_manager.py</code>           | Master miner subprocess manager. Spawns/kills XMRig and cpuminer-multi, parses stdout in real time, maintains live stats dict.     |
| <code>miner_swarm.py</code>             | SwarmEngine: spawns up to 40 SwarmNode subprocesses simultaneously (staggered 0.25s), aggregates hashrates, feeds price data.      |
| <code>wallet_core.py</code>             | BIP39/BIP32/BIP44 HD wallet: mnemonic generation, seed derivation, BTC/LTC/DASH/ETH/XMR address derivation, tx signing, broadcast. |
| <code>wallet_api.py</code>              | Flask Blueprint for all /wallet/* API routes: create, restore, balances, transfer, broadcast, address book, mining credits.        |
| <code>wallet_db.py</code>               | SQLite wallet tables: keystore, coin addresses, transaction history, mining credit ledger, address book, portfolio cache.          |
| <code>wallet_validator.py</code>        | Background pre-flight validator. Checks pool reachability, wallet address format, and config completeness before mining starts.    |
| <code>ump_hardware_detect.py</code>     | Hardware capability engine: CPU model/cores, GPU detection (nvidia-smi, rocm-smi), arch, platform, RAM, builds capability profile. |
| <code>ump_miner_registry.py</code>      | Extended coin/algo/miner/pool/command mapping. Covers CPU, GPU (NVIDIA/AMD), and unMineable proxied coins with full binary args.   |
| <code>ump_profit_switcher.py</code>     | APScheduler profit-switching daemon. Polls CoinGecko + WhatToMine, scores coins vs hardware, auto-switches if gain >= threshold.   |
| <code>ump_universal_installer.py</code> | Python-based universal installer. Cross-platform dependency resolution and XMRig source build orchestration.                       |
| <code>miner_config.py</code>            | (Also) VERSION, ARCH, PLATFORM constants; hashrate formatter; earnings calculator; algo switcher.                                  |
| <code>unicorn_config.py</code>          | Gunicorn production config: gthread workers, SSE-safe timeout=0, ARM-tuned thread count, access/error log paths.                   |

| FILE                       | PURPOSE                                                                                                          |
|----------------------------|------------------------------------------------------------------------------------------------------------------|
| install_universal_miner.sh | Bash installer: OS/arch detect, system packages (cmake/git/gcc/openssl/libuv), XMRig source clone + cmake build. |
| install_ump_universal.sh   | Secondary bash installer with extended platform support and Python venv setup.                                   |
| start.sh / stop.sh         | Foreground start and graceful stop scripts.                                                                      |
| start_screen.sh            | Background start via GNU screen session named 'ump_miner'.                                                       |
| start_gunicorn.sh          | Production start via Gunicorn using gunicorn_config.py.                                                          |
| wsgi.py                    | WSGI entry for Gunicorn: imports app from universal_miner.                                                       |
| requirements.txt           | Python deps: flask, requests, psutil, apscheduler, gunicorn, cryptography.                                       |
| templates/dashboard.html   | Live mining dashboard: stats, hashrate chart, session log, swarm status, profit switcher controls.               |
| templates/setup.html       | Profile setup wizard: coin/pool selection, credentials, hardware config, wallet config.                          |
| templates/profiles.html    | Profile manager: list, select active, export JSON, delete.                                                       |
| templates/wallet_ui.html   | Full HD wallet UI: create/restore, balances, send/receive, transaction history, address book.                    |
| bin/                       | Binary directory: xmrig (built by installer), optional GPU miners (t-rex, lolminer, teamredminer, gminer).       |
| logs/                      | Per-profile XMRig logs, Gunicorn access/error logs, profit switcher log.                                         |
| miner.db                   | SQLite database (auto-created). All profiles, sessions, wallet data, price cache.                                |

---

## 3. CORE MODULES - DETAILED BREAKDOWN

---

### 3.1 universal\_miner.py - Application Core

The Flask application root. Initialises both the miner database and wallet database at startup, registers all blueprints (swarm, wallet, hardware, registry, profit), spawns background threads for hardware detection and wallet validation, and defines the primary HTTP routes including SSE streaming and the health endpoint.

Key responsibilities:

- Flask app factory with random 24-byte secret key per process
- `init_db()` + `init_wallet_db()` called before first request
- `start_background_validation()` launched in daemon thread
- Hardware capability profile detected non-blocking in background thread
- `register_with_app(app)` wires SwarmEngine blueprint at `/api/swarm/*`
- `register_wallet(app)` wires HD Wallet blueprint at `/wallet/*`
- Conditional module loading: hardware detect, registry, profit switcher all wrapped in try/except so missing optional deps don't crash startup
- SSE `/stream` endpoint yields JSON stats every 2 seconds, compatible with EventSource in all browsers
- `/health` endpoint returns JSON uptime + version for load balancer checks

### 3.2 miner\_manager.py - Mining Process Manager

Manages the single master miner subprocess. Provides start/stop/status functions that are called by Flask routes. Uses `threading.Lock` for all shared state to be thread-safe under Gunicorn's gthread workers.

State tracking:

- `_proc`: the active subprocess.Popen object
- `_stats` dict: running, algo, coin, pool, hashrate, hashrate\_unit, accepted, rejected, uptime\_sec, started\_at, profile\_name, pid, error, daily\_usd, efficiency, temp, cpu\_pct
- `_session_id`: FK into sessions table for current mining session
- `_log_thread`: stdout reader thread, one line per iteration
- `_heartbeat_thread`: writes uptime + CPU% to stats every 5 seconds

Output parsers:

- `_parse_xmrig()`: regex for 'speed 10s/60s/15m X.X H/s', 'accepted (N/M)', temperature
- `_parse_cpuminer()`: regex for 'yay!!! X.XXX MH/s', accepted/rejected blocks
- Auto-detects parser from current algo\_key (RANDOMX -> xmrig, X11 -> cpuminer)
- `auto_resume()` called at startup: if miner\_state table says was running, relaunch
- `switch_algo()` hot-swaps binary + config without stopping the Flask server

### 3.3 miner\_swarm.py - 40-Node Swarm Engine

The most complex module. Implements a SwarmNode class (one real OS subprocess each) and SwarmEngine (controller for all nodes + price feed + hardware monitor). Total mining

---

processes when swarm is active: 40 swarm nodes + 1 master = 41 processes.

SwarmNode class:

- One XMRig or cpuminer-multi subprocess per node, launched via subprocess.Popen
- Dedicated stdout reader thread per node parsing hashrate + shares in real time
- on\_update callback fires on every stat change, updating SwarmEngine shared state
- Independent stop\_evt per node for clean shutdown without killing other nodes
- node\_id 1..40 used for log file naming and API identification

SwarmEngine class:

- shared dict: aggregate hashrate, total accepted, alive node count, per-node stats
- start\_swarm(profile, threads, algo): staggered launch (0.25s between nodes) to avoid CPU spike on weak hardware
- stop\_swarm(): signals all node stop\_evts, waits for thread join, kills any remaining proc
- Price feed thread: polls CoinGecko /simple/price every 2 minutes, caches USD rates
- Hardware monitor thread: polls psutil every 30s for CPU%, memory%, disk usage
- snapshot\_history: ring buffer of last 60 aggregate stat snapshots
- Flask Blueprint /api/swarm/\* with endpoints: status, start, stop, prices, hardware, node/N



### 3.4 wallet\_core.py - HD Blockchain Wallet

A full BIP39/BIP32/BIP44 hierarchical deterministic wallet implemented entirely in Python standard library plus the 'cryptography' package. No external wallet libraries. Supports real address generation, real ECDSA transaction signing, and real blockchain broadcast.

| FEATURE                | DETAIL                                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------|
| BIP39 Mnemonic         | 2048-word embedded wordlist. Generates 12 or 24-word mnemonics from os.urandom entropy. Full checksum validation.        |
| BIP32 Master Key       | HMAC-SHA512 seed -> master key derivation. Child key derivation (hardened and normal) with secp256k1 ECDSA.              |
| BIP44 Paths            | m/44'/0'/0'/0/0 (BTC), m/44'/2'/0'/0/0 (LTC), m/44'/5'/0'/0/0 (DASH), m/44'/60'/0'/0/0 (ETH)                             |
| BTC Addresses          | P2PKH (legacy 1...) and P2WPKH (native segwit bc1...) via bech32 encoding                                                |
| LTC Addresses          | P2PKH (L...) and P2WPKH (ltc1...) with Litecoin version bytes                                                            |
| DASH Addresses         | P2PKH with DASH version byte 0x4C -> X... prefix                                                                         |
| ETH Addresses          | Keccak-256 of uncompressed pubkey, EIP-55 checksum encoding                                                              |
| XMR Addresses          | Ed25519 key derivation via sc_reduce32, base58 Monero encoding with checksum                                             |
| TX Signing (BTC)       | Full UTXO fetch from blockstream.info, P2PKH scriptSig construction, DER-encoded ECDSA signature, raw hex output         |
| TX Broadcast           | HTTP POST to blockstream.info/api/tx for BTC, analogous explorers for LTC/DASH                                           |
| Balance Queries        | blockstream.info (BTC), blockchair (LTC/DASH), etherscan.io (ETH), moneroworld/getmonero (XMR)                           |
| AES-256-GCM Encryption | Private keys and mnemonics stored encrypted with AESGCM using a key derived from the wallet name + machine-specific salt |
| Binance Integration    | HMAC-SHA256 signed API requests to Binance /sapi/v1/capital/deposit/address and /sapi/v1/capital/withdraw/apply          |

### 3.5 ump\_profit\_switcher.py - Autonomous Profit Daemon

An APScheduler BackgroundScheduler job that runs every 5 minutes (configurable). It fetches live USD prices from CoinGecko, queries WhatToMine for network difficulty and block reward estimates, scores every coin in the registry against the detected hardware's hashrate, and issues a miner switch command if the best opportunity exceeds the current coin by  $\geq$  MIN\_SWITCH\_GAIN\_PCT (default 5%).

- PRICE\_CACHE\_TTL: 120 seconds - prices are reused within window to avoid rate limiting
- SWITCH\_INTERVAL\_SEC: 300 - evaluation frequency
- MIN\_SWITCH\_GAIN\_PCT: 5.0 - minimum profit gain to trigger switch (prevents thrashing)
- 23 coins mapped to CoinGecko IDs for live price feeds
- All switch events logged to logs/profit\_switch.log with timestamp, from/to coin, gain %

- Flask Blueprint `/api/profit/*` with endpoints: `status`, `history`, `set_threshold`, `force_eval`
- Graceful degradation: if requests library missing, daemon disables itself cleanly

### 3.6 ump\_hardware\_detect.py - Capability Engine

Probes the host system at startup and builds a structured `capability_profile` dict used by the profit switcher and UI to determine which coins/algos are viable.

| DETECTION        | METHOD                                                                                                                                                            |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform         | Filesystem markers: <code>/data/data/com.termux</code> (Termux), <code>/proc/version</code> 'userland' (Userland Ubuntu), <code>platform.system()</code> fallback |
| Architecture     | <code>platform.machine()</code> : <code>armv7l</code> , <code>aarch64</code> , <code>x86_64</code> , <code>x86</code>                                             |
| CPU Model        | Parses <code>/proc/cpuinfo</code> 'model name' on Linux; <code>sysctl machdep.cpu.brand_string</code> on macOS; <code>wmic</code> on Windows                      |
| CPU Cores        | <code>os.cpu_count()</code> with <code>psutil</code> fallback                                                                                                     |
| RAM              | <code>psutil.virtual_memory()</code> total and available                                                                                                          |
| NVIDIA GPU       | <code>subprocess nvidia-smi --query-gpu=name,memory.total --format=csv,noheader</code>                                                                            |
| AMD GPU          | <code>subprocess rocm-smi --showproductname</code>                                                                                                                |
| Capability Tag   | <code>CPU_ONLY</code> / <code>GPU_ONLY</code> / <code>CPU_GPU</code> based on detection results                                                                   |
| Mining Viability | Returns list of viable algo keys based on detected hardware class                                                                                                 |

## 4. SUPPORTED HARDWARE & PLATFORMS

| PLATFORM         | ARCH            | MINING TYPE   | NOTES                                                                          |
|------------------|-----------------|---------------|--------------------------------------------------------------------------------|
| Android / Termux | armv7l          | CPU (RandomX) | XMRig built from source. 1-2 threads max on weak chips. Build takes 15-25 min. |
| Android / Termux | aarch64         | CPU (RandomX) | Full 4-8 core usage viable on flagship SoCs (SD 8xx, Dimensity 9xxx).          |
| Linux Desktop    | x86_64          | CPU + GPU     | Full swarm capability. GPU miners placed in bin/ manually.                     |
| Linux Server     | x86_64 / arm64  | CPU (RandomX) | Headless-friendly. Screen session or Unicorn for persistence.                  |
| macOS            | x86_64 / M-chip | CPU (RandomX) | Homebrew required for build deps. M-chip: ARM64 XMRig builds natively.         |
| Windows          | x86_64          | CPU (RandomX) | WSL recommended. Native support via subprocess.                                |
| Raspberry Pi     | armv7l / arm64  | CPU (RandomX) | Very low hashrate (~50-200 H/s). Useful for learning / demo.                   |
| Userland Ubuntu  | armv7l / arm64  | CPU (RandomX) | Francois's primary target. Detected via /proc/version 'userland' marker.       |

### GPU Miner Support

| GPU BRAND    | RECOMMENDED BINARY | SUPPORTED ALGORITHMS                    |
|--------------|--------------------|-----------------------------------------|
| NVIDIA       | T-Rex              | KawPow (RVN, CLORE, XNA), Octopus (CFX) |
| NVIDIA       | lolMiner           | Ethash (ETC), Equihash (ZEC)            |
| AMD          | TeamRedMiner       | KawPow, Ethash (ETC), AutoLykos2 (ERG)  |
| NVIDIA + AMD | GMiner             | Equihash, BeamHash, Octopus             |

GPU miner binaries must be manually placed in bin/ and named 'lolminer', 't-rex', 'teamredminer', or 'gminer'. The dashboard tracks GPU miners via process detection but does not manage their configuration files directly.

## 5. COIN REGISTRY - ALL SUPPORTED CRYPTOCURRENCIES

UMP supports 30+ coins across CPU, GPU, and ASIC hardware classes. The full registry is defined in `miner_config.py` (COINS dict) and extended by `ump_miner_registry.py` with full binary argument mapping.

### CPU-Minable Coins (RandomX / X11)

| TICKER | NAME            | ALGORITHM   | POOL SUPPORT                              |
|--------|-----------------|-------------|-------------------------------------------|
| XMR    | Monero          | RandomX     | Kryptex, ViaBTC, MoneroOcean, MPH, Direct |
| ZEPH   | Zephyr Protocol | RandomX     | Kryptex, MoneroOcean                      |
| SAL    | Salvium         | RandomX     | Kryptex, MoneroOcean                      |
| DOGE   | Dogecoin        | RandomX     | unMineable only (proxied payout)          |
| SHIB   | Shiba Inu       | RandomX     | unMineable only (proxied payout)          |
| ADA    | Cardano         | RandomX     | unMineable only (proxied payout)          |
| SOL    | Solana          | RandomX     | unMineable only (proxied payout)          |
| BNB    | Binance Coin    | RandomX     | unMineable only (proxied payout)          |
| TRX    | Tron            | RandomX     | unMineable only (proxied payout)          |
| XRP    | Ripple          | RandomX     | unMineable only (proxied payout)          |
| MATIC  | Polygon         | RandomX     | unMineable only (proxied payout)          |
| DOT    | Polkadot        | RandomX     | unMineable only (proxied payout)          |
| LINK   | Chainlink       | RandomX     | unMineable only (proxied payout)          |
| AVAX   | Avalanche       | RandomX     | unMineable only (proxied payout)          |
| DASH   | Dash            | X11         | Binance Pool, Direct                      |
| RTM    | Raptoreum       | GhostRider  | Direct / unMineable                       |
| WOW    | Wownero         | RandomX/WOW | Direct / MoneroOcean                      |

### GPU-Minable Coins

| TICKER | NAME             | ALGORITHM  | GPU AFFINITY                       |
|--------|------------------|------------|------------------------------------|
| RVN    | Ravencoin        | KawPow     | NVIDIA T-Rex / AMD TeamRedMiner    |
| XNA    | Neurai           | KawPow     | NVIDIA T-Rex / AMD TeamRedMiner    |
| CLORE  | Clore.ai         | KawPow     | NVIDIA T-Rex                       |
| ETC    | Ethereum Classic | Etchash    | NVIDIA lolMiner / AMD TeamRedMiner |
| ETHW   | EthereumPoW      | Ethash     | NVIDIA lolMiner / AMD TeamRedMiner |
| OCTA   | OctaSpace        | Ethash     | NVIDIA lolMiner                    |
| ERG    | Ergo             | Autolykos2 | AMD TeamRedMiner                   |
| IRON   | Iron Fish        | IronFish   | NVIDIA / AMD                       |

| TICKER | NAME         | ALGORITHM   | GPU AFFINITY        |
|--------|--------------|-------------|---------------------|
| CFX    | Conflux      | Octopus     | NVIDIA T-Rex        |
| NEXA   | Nexa         | NexaPoW     | NVIDIA / AMD        |
| ALPH   | Alephium     | SHA3x       | NVIDIA / AMD / ASIC |
| XTM    | Tari         | RandomX/XTM | GPU + CPU           |
| RXD    | Radiant      | SHA512/256d | GPU                 |
| XEL    | Xelis        | XelisHash   | GPU                 |
| QUAI   | Quai Network | ProgPoW     | GPU                 |

## ASIC-Minable Coins

| TICKER | NAME            | ALGORITHM | ASIC AFFINITY                               |
|--------|-----------------|-----------|---------------------------------------------|
| BTC    | Bitcoin         | SHA-256   | All SHA-256 ASICs (Antminer S19/S21 series) |
| BCH    | Bitcoin Cash    | SHA-256   | All SHA-256 ASICs                           |
| BSV    | Bitcoin SV      | SHA-256   | All SHA-256 ASICs                           |
| FB     | Fractal Bitcoin | SHA-256   | All SHA-256 ASICs                           |
| LTC    | Litecoin        | Scrypt    | Scrypt ASICs (Antminer L7)                  |
| KAS    | Kaspa           | HeavyHash | Kaspa ASICs (IceRiver, Bitmain)             |
| ZEC    | Zcash           | Equihash  | Equihash ASICs (Antminer Z15)               |
| XEC    | eCash           | SHA-256   | All SHA-256 ASICs                           |
| DGB    | DigiByte        | SHA-256   | Multi-algo; SHA-256 ASIC channel            |

---

## 6. MINING POOL INTEGRATION

---

### **Kryptex Pool [ [pool.kryptex.network](https://pool.kryptex.network) ]**

7 named stratum endpoints (XMR, ZEPH, SAL, BTC, LTC, KAS, ZEC, ETC, RVN, ERG, ALPH). Auto-converts mined coins to BTC. Credential: email.worker / X. One of the most widely supported multi-coin pools.

### **ViaBTC [ [viabtc.com](https://viabtc.com) ]**

PPS+ payout model. Established 2016. 9 coins including BTC, BCH, LTC, ETH, ETC, ZEC, DASH, XMR, KAS. Credential: username.worker / 123.

### **unMineable [ [rx.unmineable.com:3333](https://rx.unmineable.com:3333) ]**

Unique model: mine with CPU (RandomX) and receive payout in any of 100+ coins including BTC, ETH, DOGE, ADA, SOL, SHIB. Credential format: COIN:WALLET#REFERRAL\_CODE. Password is worker name.

### **Mining Pool Hub [ [miningpoolhub.com](https://miningpoolhub.com) ]**

Auto-switching multi-algorithm pool. Requires registration and API key. Credential: mph\_username.worker / x. Best for multi-coin auto-switching setups.

### **MoneroOcean [ [gulf.moneroocean.stream:10128](https://gulf.moneroocean.stream:10128) ]**

XMR-focused pool that auto-switches RandomX algorithm variants for maximum profitability. No registration required - use XMR wallet as username. SSL port: 20128.

### **Binance Pool [ [bs.poolbinance.com:1800](https://bs.poolbinance.com:1800) ]**

Integrated with Binance exchange account. Direct payout to Binance balance. Credential: binance\_username.worker / x. Requires Binance account.

### **Direct Wallet [ custom stratum URL ]**

Any pool not listed above. Enter custom stratum URL, username, and password. Full flexibility for custom setups or new pools not yet in the registry.

## **Pool URL Auto-Fill**

The `/api/suggest_pool_url` endpoint takes a coin ticker and pool key and returns the correct stratum URL automatically. The Setup UI has an AUTO-FILL button that calls this endpoint and populates the pool URL field, eliminating manual entry errors.

---

## 7. SWARM ENGINE - 41-NODE PARALLEL MINING

---

The SwarmEngine is UMP's most powerful feature. When activated, it spawns 40 SwarmNode subprocesses (TOTAL\_NODES = 40) in addition to the master miner process, for a total of 41 simultaneous mining processes. Each node is a real OS-level subprocess running a genuine XMRig or cpuminer-multi binary.

### Architecture

- NODE\_THREADS = 1 per swarm node (configurable). Total CPU threads = 41 x NODE\_THREADS.
- STAGGER\_SEC = 0.25 seconds between node launches to avoid memory allocation spikes.
- Each node writes to its own log file: logs/swarm\_node\_N.log
- Aggregate hashrate = sum of all alive node hashrates + master hashrate
- Dead nodes (crash/OOM) are tracked in shared['dead\_nodes'] and reported via API
- SwarmEngine runs its own price feed thread independently of the profit switcher

### Swarm API Endpoints

| ENDPOINT             | METHOD | DESCRIPTION                                                   |
|----------------------|--------|---------------------------------------------------------------|
| /api/swarm/status    | GET    | Aggregate hashrate, alive nodes, per-node stats, price data   |
| /api/swarm/start     | POST   | Start all 40 swarm nodes. Body: {profile_name, threads, algo} |
| /api/swarm/stop      | POST   | Stop all swarm nodes cleanly                                  |
| /api/swarm/prices    | GET    | Live coin prices from CoinGecko cache                         |
| /api/swarm/hardware  | GET    | CPU%, memory%, temperature from psutil                        |
| /api/swarm/node/{id} | GET    | Individual node stats: hashrate, accepted, alive, error       |
| /api/swarm/history   | GET    | Last 60 aggregate snapshots (ring buffer)                     |
| /api/swarm/earnings  | GET    | Estimated daily USD based on aggregate hashrate + live price  |

### Performance Expectations

Expected aggregate hashrate varies dramatically by hardware. On weak ARM hardware (armv7l single-core), running 40 nodes with 1 thread each is CPU-bound and will likely be less efficient than 1 node with N threads. The swarm architecture is most beneficial on high-core-count x86\_64 servers where 40 x 1-thread nodes can saturate all physical cores without the coordination overhead of XMRig's internal thread manager.

---

## 8. DUAL-ALGORITHM SYSTEM

---

UMP supports two CPU mining algorithms simultaneously configured and switchable at runtime without restarting the Flask server.

| PROPERTY           | RANDOMX                                       | X11                                         |
|--------------------|-----------------------------------------------|---------------------------------------------|
| Binary             | xmrig                                         | cpuminer-multi                              |
| Algorithm Spec     | rx/0                                          | x11                                         |
| Hashrate Unit      | H/s (hashes/sec)                              | MH/s (megahashes/sec)                       |
| Primary Coin       | XMR, ZEPH, SAL                                | DASH (direct) / any via unMineable          |
| CPU Optimization   | L3 cache intensive, NUMA-aware, AES-NI        | Chained 11 hash functions, lighter on cache |
| Memory Requirement | ~2 MB L3 per thread (large pages preferred)   | ~64 MB RAM per instance                     |
| ASIC Resistance    | Yes - designed to be ASIC-resistant           | No - DASH ASICs exist                       |
| Pool Compatibility | Kryptex, MoneroOcean, ViaBTC, MPH, unMineable | Binance Pool, unMineable, Direct            |
| Switch Command     | /api/algo/set {algo: RANDOMX}                 | /api/algo/set {algo: X11}                   |

### Hot Algorithm Switching

The `switch_algo()` function in `miner_manager.py` gracefully stops the active miner, updates the `ALGOS['active']` pointer in `miner_config.py`, selects the correct binary path from `BIN_DIR`, rebuilds the command line, and relaunches the process. The Flask server does not restart. Stats are reset. Session history records the switch event.



## 9. HD WALLET SYSTEM - FULL BLOCKCHAIN WALLET

UMP includes a complete self-custodial hierarchical deterministic (HD) wallet that requires no third-party wallet software. It generates real, spendable blockchain addresses, signs real transactions, and broadcasts them directly to blockchain networks. This is not a mock or simulation - it implements the actual BIP standards used by hardware wallets and production software wallets.

### Supported Chains

| CHAIN          | STANDARD               | ADDRESS FORMATS                 | BALANCE API      |
|----------------|------------------------|---------------------------------|------------------|
| Bitcoin (BTC)  | BIP44 m/44'/0'/0'/0/0  | P2PKH (1...) + P2WPKH (bc1...)  | blockstream.info |
| Litecoin (LTC) | BIP44 m/44'/2'/0'/0/0  | P2PKH (L...) + P2WPKH (ltc1...) | blockchair.com   |
| Dash (DASH)    | BIP44 m/44'/5'/0'/0/0  | P2PKH (X...)                    | blockchair.com   |
| Ethereum (ETH) | BIP44 m/44'/60'/0'/0/0 | EIP-55 checksummed hex (0x...)  | etherscan.io     |
| Monero (XMR)   | Ed25519 / Monero spec  | 95... standard address          | moneroworld.com  |

### Wallet Operations

| OPERATION            | ENDPOINT                         | DESCRIPTION                                                                                     |
|----------------------|----------------------------------|-------------------------------------------------------------------------------------------------|
| Create Wallet        | /wallet/api/create               | Generate new 12-word BIP39 mnemonic, derive all chain addresses, encrypt and store in wallet_db |
| Restore Wallet       | /wallet/api/restore              | Restore from existing mnemonic + optional passphrase, re-derive all addresses                   |
| List Wallets         | /wallet/api/list                 | All wallet names in local keystore                                                              |
| Get Addresses        | /wallet/api/addresses            | All derived addresses for active wallet                                                         |
| Fetch Balances       | /wallet/api/balances             | Live blockchain queries for all chains, cached for 5 min                                        |
| Portfolio Summary    | /wallet/api/portfolio            | Total portfolio value in USD using CoinGecko prices                                             |
| Send Transaction     | /wallet/api/transfer             | Build, sign, and broadcast a BTC/LTC/DASH transaction                                           |
| Transfer to Exchange | /wallet/api/transfer_to_exchange | Withdraw to Binance using stored API keys                                                       |
| Transaction History  | /wallet/api/transactions         | All outgoing transactions with status tracking                                                  |

| OPERATION               | ENDPOINT                    | DESCRIPTION                                                            |
|-------------------------|-----------------------------|------------------------------------------------------------------------|
| Mining Credits          | /wallet/api/mining_credits  | Ledger of mining earnings recorded by the mining session tracker       |
| Address Book            | /wallet/api/address_book    | Save and retrieve named external addresses                             |
| Reveal Mnemonic         | /wallet/api/reveal_mnemonic | Decrypt and return mnemonic (requires confirmation)                    |
| Reveal Private Key      | /wallet/api/reveal_privkey  | Decrypt and return per-coin private key                                |
| Binance Deposit Address | /wallet/api/binance_deposit | Fetch Binance deposit address for specified coin using stored API keys |

## Security Model

- Mnemonics and private keys stored encrypted with AES-256-GCM
- Encryption key derived from wallet name + machine-specific salt (never stored)
- Flask session cookie (random 24-byte secret per process) used to track active wallet
- Read-only Binance API keys recommended - trading keys should never be entered
- XMR view key allows balance monitoring only - spend key never required by UMP
- wallet.db stored locally - never transmitted externally
- reveal\_mnemonic and reveal\_privkey endpoints require explicit user confirmation

## 10. PROFIT SWITCHING DAEMON

The autonomous profit switching daemon continuously monitors cryptocurrency market conditions and automatically switches the active mining coin/algorithm to maximise USD earnings per unit of hardware capacity. It runs as a non-blocking APScheduler BackgroundScheduler job, entirely independent of the Flask request/response cycle.

### Decision Algorithm

- Step 1: Fetch live USD prices for all registered coins from CoinGecko API
- Step 2: Query WhatToMine for estimated daily revenue per coin at standard hashrate
- Step 3: Scale WhatToMine estimates to actual detected hardware hashrate
- Step 4: Score each coin:  $\text{score} = \text{estimated\_daily\_usd} * \text{price\_confidence\_factor}$
- Step 5: Compare `best_coin.score` vs `current_coin.score`
- Step 6: If  $(\text{best} - \text{current}) / \text{current} \geq \text{MIN\_SWITCH\_GAIN\_PCT}$  -> trigger switch
- Step 7: Call `switch_algo()` + restart miner with new profile
- Step 8: Log event with timestamp, coins, gain% to SQLite and log file

### Supported Coins for Auto-Switch

The following 23 coins are tracked with CoinGecko IDs for live price data: XMR, ZEPH, SAL, DOGE, ADA, SOL, BNB, TRX, XRP, MATIC, DOT, LINK, AVAX, RVN, ETC, ERG, KAS, ALPH, BTC, LTC, BCH, RTM, WOW.

### Configuration Parameters

| PARAMETER           | DEFAULT                | DESCRIPTION                                                  |
|---------------------|------------------------|--------------------------------------------------------------|
| PRICE_CACHE_TTL     | 120 sec                | How long CoinGecko prices are valid before re-fetch          |
| SWITCH_INTERVAL_SEC | 300 sec                | How often the daemon evaluates profitability                 |
| MIN_SWITCH_GAIN_PCT | 5.0%                   | Minimum profit improvement required to trigger a coin switch |
| LOG_PATH            | logs/profit_switch.log | Switch event log file                                        |
| DB_PATH             | miner.db               | SQLite database for switch history                           |

# 11. HARDWARE DETECTION ENGINE

At application startup, `ump_hardware_detect.py` probes the host system in a background thread (non-blocking) and builds a `capability_profile` dict that is served via `/api/hardware` and used by the profit switcher to filter applicable coins.

## Detection Methods

| TARGET           | METHOD                                                          | OUTPUT                                             |
|------------------|-----------------------------------------------------------------|----------------------------------------------------|
| OS Platform      | Filesystem + <code>platform.system()</code>                     | termux / userland_ubuntu / linux / macos / windows |
| CPU Architecture | <code>platform.machine()</code>                                 | armv7l / aarch64 / x86_64 / x86                    |
| CPU Model Name   | <code>/proc/cpuinfo</code> (Linux), <code>sysctl</code> (macOS) | Full model string e.g. 'Snapdragon 888'            |
| Physical Cores   | <code>os.cpu_count()</code> + <code>psutil</code> fallback      | Integer core count                                 |
| Total RAM        | <code>psutil.virtual_memory().total</code>                      | Bytes, formatted as GB                             |
| Available RAM    | <code>psutil.virtual_memory().available</code>                  | Bytes, used to warn low-memory nodes               |
| NVIDIA GPU       | <code>nvidia-smi --query-gpu=name,memory</code>                 | GPU name + VRAM if <code>nvidia-smi</code> present |
| AMD GPU          | <code>rocm-smi --showproductname</code>                         | GPU name if ROCm installed                         |
| Capability Class | Derived from CPU+GPU detection                                  | CPU_ONLY / GPU_ONLY / CPU_GPU                      |
| Viable Algos     | Capability class -> algo list                                   | e.g. [RANDOMX] for CPU_ONLY                        |

The capability profile is cached in memory after first detection. Re-detection can be triggered via the `/api/hardware/refresh` endpoint without restarting the application.

## 12. REST API - COMPLETE ENDPOINT REFERENCE

### Core Mining Endpoints

| METHOD | ENDPOINT          | DESCRIPTION                                                                                 |
|--------|-------------------|---------------------------------------------------------------------------------------------|
| GET    | /                 | Main dashboard HTML page                                                                    |
| GET    | /setup            | Setup wizard HTML page                                                                      |
| GET    | /profiles         | Profile manager HTML page                                                                   |
| POST   | /mine/start       | Start master miner. Form: profile_name, threads                                             |
| POST   | /mine/stop        | Stop master miner gracefully                                                                |
| GET    | /api/stats        | Live miner stats JSON: running, hashrate, coin, pool, accepted, rejected, uptime, daily_usd |
| GET    | /api/log?lines=80 | Last N lines of active XMRig/cpuminer log                                                   |
| GET    | /stream           | SSE stream: pushes stats JSON every 2 seconds                                               |
| GET    | /health           | Health check: {status: ok, version, uptime}                                                 |

### Profile & Config Endpoints

| METHOD | ENDPOINT              | DESCRIPTION                                            |
|--------|-----------------------|--------------------------------------------------------|
| GET    | /api/profiles         | List all saved profiles as JSON array                  |
| POST   | /api/profiles         | Create/update profile. Body: full profile dict         |
| DELETE | /api/profiles/{name}  | Delete named profile                                   |
| GET    | /api/coins            | Full coin registry JSON                                |
| GET    | /api/pools            | Full pool registry JSON                                |
| GET    | /api/algos            | Algorithm registry JSON                                |
| GET    | /api/suggest_pool_url | Auto pool URL. Params: coin, pool                      |
| GET    | /api/hashrate_calc    | Earnings estimate. Params: hashrate, unit, coin, price |
| GET    | /api/sysinfo          | Platform/arch/version info                             |
| GET    | /api/sessions         | Mining session history from SQLite                     |

### Swarm Endpoints

| METHOD | ENDPOINT            | DESCRIPTION                                                          |
|--------|---------------------|----------------------------------------------------------------------|
| GET    | /api/swarm/status   | Full swarm status: aggregate hashrate, alive nodes, prices, hardware |
| POST   | /api/swarm/start    | Start all 40 swarm nodes                                             |
| POST   | /api/swarm/stop     | Stop all swarm nodes                                                 |
| GET    | /api/swarm/prices   | CoinGecko price cache                                                |
| GET    | /api/swarm/hardware | Live psutil hardware metrics                                         |

| METHOD | ENDPOINT             | DESCRIPTION                 |
|--------|----------------------|-----------------------------|
| GET    | /api/swarm/node/{id} | Individual swarm node stats |

## Hardware & Registry Endpoints

| METHOD | ENDPOINT              | DESCRIPTION                                     |
|--------|-----------------------|-------------------------------------------------|
| GET    | /api/hardware         | Detected hardware capability profile            |
| GET    | /api/hardware/refresh | Re-run hardware detection                       |
| GET    | /api/registry/coins   | Extended coin registry from ump_miner_registry  |
| GET    | /api/registry/algos   | Extended algorithm registry                     |
| GET    | /api/registry/miners  | Available miner binaries and their capabilities |
| GET    | /api/registry/pools   | Extended pool registry with full stratum URLs   |

## Wallet Endpoints

| METHOD | ENDPOINT                           | DESCRIPTION                          |
|--------|------------------------------------|--------------------------------------|
| GET    | /wallet/                           | HD Wallet UI dashboard               |
| POST   | /wallet/api/create                 | Create new HD wallet                 |
| POST   | /wallet/api/restore                | Restore wallet from mnemonic         |
| POST   | /wallet/api/select                 | Set active wallet                    |
| GET    | /wallet/api/list                   | List all wallets                     |
| POST   | /wallet/api/delete                 | Delete a wallet                      |
| GET    | /wallet/api/portfolio              | Total portfolio USD value            |
| GET    | /wallet/api/balances               | Live balances for all chains         |
| GET    | /wallet/api/prices                 | Coin prices in USD                   |
| GET    | /wallet/api/addresses              | All derived addresses                |
| GET    | /wallet/api/address/{coin}         | Specific coin address                |
| GET    | /wallet/api/binance_deposit/{coin} | Binance deposit address              |
| POST   | /wallet/api/transfer               | Build + sign + broadcast transaction |
| POST   | /wallet/api/transfer_to_exchange   | Withdraw to Binance                  |
| GET    | /wallet/api/transactions           | Transaction history                  |
| GET    | /wallet/api/mining_credits         | Mining earnings ledger               |
| GET    | /wallet/api/address_book           | Saved addresses                      |
| POST   | /wallet/api/address_book/save      | Save address                         |
| POST   | /wallet/api/address_book/delete    | Delete saved address                 |
| POST   | /wallet/api/record_mining          | Record a mining credit               |
| POST   | /wallet/api/reveal_mnemonic        | Decrypt and return mnemonic          |
| POST   | /wallet/api/reveal_privkey         | Decrypt and return private key       |

## 13. DATABASE SCHEMA & STORAGE

All persistent data is stored in a single SQLite file: miner.db. Two init functions create the schema: `init_db()` for mining tables and `init_wallet_db()` for wallet tables. Both use `CREATE TABLE IF NOT EXISTS`, so re-running after an upgrade is safe.

### Mining Tables

| TABLE           | KEY COLUMNS                                                                                                                                                                                                                                                 | PURPOSE                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| profiles        | name (PK), coin, pool, pool_url, miner_user, miner_pass, tls, threads, algo, kryptex_email, viabtc_user, unmineable_wallet, mph_user, moneroocean_wallet, binance_user, xmr_wallet, btc_wallet, binance_api_key, binance_api_secret, created_at, updated_at | Complete mining profile storage. One row per saved profile. |
| sessions        | id (PK), profile_name, coin, pool, algo, started_at, ended_at, duration_sec, avg_hashrate, peak_hashrate, accepted, rejected, daily_usd_est                                                                                                                 | Historical record of every mining session.                  |
| miner_state     | id (PK), profile_name, algo, was_running, last_updated                                                                                                                                                                                                      | Persists last known state for auto-resume on app restart.   |
| price_cache     | coin (PK), price_usd, updated_at                                                                                                                                                                                                                            | CoinGecko price cache to reduce API calls.                  |
| swarm_snapshots | id (PK), timestamp, total_hashrate, alive_nodes, aggregate_json                                                                                                                                                                                             | Ring buffer of swarm aggregate stats snapshots.             |
| profit_switches | id (PK), timestamp, from_coin, to_coin, from_daily_usd, to_daily_usd, gain_pct                                                                                                                                                                              | Log of all automatic profit switch events.                  |

### Wallet Tables

| TABLE               | KEY COLUMNS                                                                    | PURPOSE                                                     |
|---------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------|
| wallet_keystore     | wallet_name (PK), encrypted_mnemonic, encrypted_privkeys_json, created_at      | AES-256-GCM encrypted mnemonic and private keys per wallet. |
| wallet_coins        | wallet_name, coin, address, derivation_path                                    | Derived blockchain addresses for each coin.                 |
| wallet_transactions | id (PK), wallet_name, coin, txid, direction, amount, fee, status, broadcast_at | Outgoing transaction history with status tracking.          |
| mining_credits      | id (PK), wallet_name, coin, amount, source_profile, credited_at                | Ledger of mining earnings recorded by the session tracker.  |

| TABLE               | KEY COLUMNS                                            | PURPOSE                                                                                  |
|---------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------|
| wallet_address_book | id (PK), wallet_name, coin, label, address, created_at | Named saved external addresses per wallet.                                               |
| portfolio_cache     | wallet_name (PK), portfolio_json, updated_at           | Cached portfolio valuation to avoid repeated balance queries.                            |
| balance_cache       | wallet_name, coin, balance, updated_at                 | Per-coin balance cache with configurable TTL.                                            |
| price_cache_w       | coin (PK), price_usd, updated_at                       | Separate price cache table for wallet module (avoids contention with miner price cache). |



---

## 14. DASHBOARD UI & FRONTEND

---

UMP ships four HTML templates, all rendered server-side by Flask's Jinja2 engine. The UI uses terminal/matrix aesthetic - dark background, green monospace text, matching the operational theme of the system.

### dashboard.html

- Live hashrate display with formatted unit (H/s / KH/s / MH/s / GH/s)
- Accepted / rejected share counters
- Active profile, coin, pool, algorithm badges
- Session uptime HH:MM:SS counter
- Estimated daily USD earnings (calculated from live price + hashrate)
- Hashrate sparkline chart (last 60 seconds, updated via SSE)
- Live XMRig log tail (last 80 lines, auto-scrolling)
- Swarm status panel: alive nodes, aggregate hashrate, dead node count
- Profit switcher status: last evaluation, current best coin, threshold
- Profile selector dropdown + START / STOP buttons
- Hashrate calculator widget: input hashrate, select coin, see daily/weekly/monthly/yearly estimates
- Hardware stats: CPU%, memory%, temperature from psutil
- SSE-powered live updates: no polling, no page refresh required

### setup.html

- Multi-tab setup wizard: General, Pool, Hardware, Wallet
- Coin selector: dropdown populated from /api/coins with algorithm badge
- Pool selector: dropdown populated from /api/pools
- AUTO-FILL button: calls /api/suggest\_pool\_url and fills stratum URL field
- Credential fields: dynamic labels update based on selected pool
- Thread count input with OS cpu\_count() as default
- Wallet address fields: XMR view wallet, BTC payout address
- Binance API key/secret fields for exchange integration
- Save Profile button: POST to /api/profiles
- Validation feedback: inline error messages from /api/validate

### profiles.html

- Table of all saved profiles: name, coin, pool, thread count, last used
- Click to activate: sets profile as active for Dashboard
- Export as JSON: downloads profile as .json backup file
- Delete with confirmation dialog
- Import from JSON: restore profile from file upload

---

## wallet\_ui.html

- Wallet selector: list + create + restore tabs
- Balance dashboard: BTC, LTC, DASH, ETH, XMR with USD value
- Portfolio total in USD with sparkline
- Send form: amount, destination address, fee selector
- Receive tab: QR-code-style text display of wallet addresses
- Transaction history table with status badges (pending/confirmed/failed)
- Mining credits ledger: shows earnings from mining sessions
- Address book: labeled saved addresses with copy button
- Settings: reveal mnemonic (confirmation required), reveal private key
- Binance: deposit address lookup, withdrawal form

# 15. INSTALLATION & DEPLOYMENT

## Quick Install (Linux / Android Termux)

```
chmod +x install_universal_miner.sh
./install_universal_miner.sh
# Installer detects OS, arch, installs build deps, clones XMRig,
# runs cmake build (15-25 min on armv7l), writes start/stop scripts.

# Start dashboard:
./start.sh # foreground
./start_screen.sh # background (GNU screen)
./start_gunicorn.sh # production (Gunicorn)

# Access:
# http://localhost:8080 (from device)
# http://DEVICE_IP:8080 (from network)
```

## Android / Termux Specific Steps

- Install Termux from F-Droid (NOT Play Store - outdated version)
- pkg update && pkg install git python cmake make gcc openssl libuv
- clone repo, chmod +x install\_universal\_miner.sh, run installer
- armv7l build: 15-25 min. Keep screen on. Device must stay awake.
- Access dashboard from phone browser at http://localhost:8080
- Access from PC on same WiFi: http://PHONE\_IP:8080
- screen -r ump\_miner to reattach to background session
- Recommended: plug in charger during mining - battery drains fast

## Gunicorn Production Deployment

For production use (persistent background server), use Gunicorn with the provided gunicorn\_config.py. Key settings tuned for ARM + SSE compatibility:

| PARAMETER        | VALUE        | REASON                                                       |
|------------------|--------------|--------------------------------------------------------------|
| bind             | 0.0.0.0:8080 | Listen on all interfaces                                     |
| worker_class     | gthread      | Required for SSE long-lived connections (sync workers block) |
| workers          | 1            | Low RAM usage on armv7l (3.6 GB total)                       |
| threads          | 4            | 4 threads handles dashboard + SSE + API simultaneously       |
| timeout          | 0            | SSE connections are long-lived - must not timeout            |
| graceful_timeout | 10 sec       | Wait for workers to finish on graceful restart               |
| preload_app      | True         | Load app code before fork - saves RAM on multi-worker        |

---

| PARAMETER | VALUE   | REASON                      |
|-----------|---------|-----------------------------|
| pidfile   | ump.pid | PID file for stop.sh to use |

## 16. CONFIGURATION REFERENCE

### miner\_config.py Constants

| CONSTANT        | DEFAULT     | DESCRIPTION                                            |
|-----------------|-------------|--------------------------------------------------------|
| VERSION         | 4.0         | Application version string                             |
| TOTAL_NODES     | 40          | Number of swarm nodes to spawn                         |
| DEFAULT_THREADS | 2           | Default miner thread count if not specified in profile |
| DEFAULT_ALGO    | RANDOMX     | Algorithm to use if not set in profile                 |
| DB_PATH         | miner.db    | SQLite database path (relative to app root)            |
| LOG_DIR         | logs/       | Directory for per-profile log files                    |
| BIN_DIR         | bin/        | Directory for miner binaries                           |
| PLATFORM        | auto-detect | Runtime value: termux/linux/macOS/windows              |
| ARCH            | auto-detect | Runtime value: armv7l/aarch64/x86_64                   |

### Profit Switcher Config

| CONSTANT            | DEFAULT                | DESCRIPTION            |
|---------------------|------------------------|------------------------|
| PRICE_CACHE_TTL     | 120 sec                | Price cache TTL        |
| SWITCH_INTERVAL_SEC | 300 sec                | Evaluation interval    |
| MIN_SWITCH_GAIN_PCT | 5.0%                   | Minimum gain to switch |
| LOG_PATH            | logs/profit_switch.log | Event log              |

### Python Dependencies

| PACKAGE      | VERSION | PURPOSE                                                   |
|--------------|---------|-----------------------------------------------------------|
| flask        | >=2.0   | Web framework, Jinja2 templates, Blueprints, SSE          |
| requests     | >=2.28  | HTTP calls to CoinGecko, WhatToMine, blockchain explorers |
| psutil       | >=5.9   | CPU%, memory%, temperature monitoring                     |
| apscheduler  | >=3.10  | BackgroundScheduler for profit switcher daemon            |
| gunicorn     | >=20.1  | Production WSGI server with gthread worker class          |
| cryptography | >=41.0  | AES-256-GCM encryption, Ed25519 key operations for wallet |

---

## 17. SECURITY ARCHITECTURE

---

UMP is designed for local network deployment with trusted users. It does not include authentication by default because it was built for single-user personal mining setups. For any internet-facing deployment, nginx + basic auth should be added as a reverse proxy.

### Local Data Only

All credentials, profiles, and wallet data are stored in miner.db on the local filesystem. Nothing is transmitted to Anthropic, the UMP developers, or any external service except legitimate API calls (pool stratums, CoinGecko, blockchain explorers, Binance).

### Wallet Encryption

Mnemonics and private keys are encrypted with AES-256-GCM before storage. The encryption key is derived at runtime from the wallet name and a machine-specific salt. The plaintext key is never persisted to disk.

### API Key Safety

Only read-only Binance API keys should be used. The system can fetch deposit addresses and initiate withdrawals - it does NOT require or support trading/order keys. API keys are stored in the profiles table but never logged.

### XMRig Source Build

XMRig is compiled from source during installation. This ensures the binary matches the target architecture and has not been tampered with. The source is cloned directly from the official XMRig GitHub repository.

### No External Authentication Service

There is no OAuth, no external auth server, no API keys required to run UMP itself. The Flask secret\_key is regenerated randomly on every process start (os.urandom(24)).

### Recommended Production Hardening

Add nginx reverse proxy with basic auth. Bind to localhost and use nginx to expose port 8080. Use HTTPS with a self-signed certificate for the local network. Run the process as a dedicated non-root user.

### Port & Interface

Default bind: 0.0.0.0:8080. This exposes the dashboard to all network interfaces. Change to 127.0.0.1:8080 in gunicorn\_config.py if only local access is needed.

---

## 18. ROADMAP & FUTURE CAPABILITIES

---

The following capabilities are architecturally planned or partially scaffolded in the v4.0 codebase and represent the natural evolution of the platform.

### **v4.1 - GPU Auto-Configuration**

Automatic T-Rex / lolMiner / TeamRedMiner binary detection and configuration. Currently GPU miners must be placed manually in bin/. v4.1 will detect GPU brand via the hardware engine and auto-configure the correct binary + coin arguments.

### **v4.1 - Remote Node Aggregation**

The swarm architecture is built for multi-machine aggregation. v4.1 will add network discovery (LAN broadcast) so multiple UMP instances can aggregate their hashrates into a single unified dashboard view.

### **v4.2 - Android Java Port (Zeus Project)**

The entire Python/Flask stack is being converted to Java for native Android APK deployment (com.zeus.android). NanoHTTPD replaces Flask, Android SQLite APIs replace the Python sqlite3 module, HttpURLConnection replaces requests. Target: inject compiled Smali bytecode into a cleaned APK via apktoolM.

### **v4.2 - Thermal Throttle Management**

Auto-reduce thread count when CPU temperature exceeds configurable threshold. The hardware engine already reads temperature via psutil/sensors. The throttle logic will be added to the heartbeat thread in miner\_manager.py.

### **v4.3 - Pool Failover**

Automatic pool switching on connection failure. If the primary pool stratum is unreachable for N consecutive seconds, switch to a secondary pool from the profile. The validator module already checks pool reachability - v4.3 adds the runtime failover.

### **v4.3 - Mining Pool API Integration**

Pull hashrate, balance, and worker stats directly from pool APIs (Kryptex, ViaBTC, MoneroOcean all have REST APIs). This allows the dashboard to show pool-side stats rather than just local XMRig output.

### **v5.0 - Autonomous Strategy Engine**

An AI-powered layer (planned to use Groq/Anthropic/OpenRouter LLM APIs) that analyzes market conditions, network difficulty trends, and hardware efficiency to generate and execute mining strategy recommendations automatically. This is the convergence point with the Zeus AI platform.

### **v5.0 - DeFi Yield Routing**

Automatically route mining proceeds from the HD wallet to yield-bearing DeFi protocols (Aave, Compound, Curve) based on APY comparisons. Mined BTC -> bridge to WBTC -> deposit





## 19. TECHNICAL APPENDIX

### A. XMRig Pool URL Reference

| COIN | KRYPTEX STRATUM           | ALGORITHM  |
|------|---------------------------|------------|
| XMR  | xmr.kryptex.network:8029  | RandomX    |
| ZEPH | zeph.kryptex.network:8029 | RandomX    |
| SAL  | sal.kryptex.network:8029  | RandomX    |
| BTC  | btc.kryptex.network:3333  | SHA-256    |
| LTC  | ltc.kryptex.network:3333  | Scrypt     |
| KAS  | kas.kryptex.network:3333  | HeavyHash  |
| ZEC  | zec.kryptex.network:7042  | Equihash   |
| ETC  | etc.kryptex.network:8888  | Etchash    |
| RVN  | rvn.kryptex.network:3333  | KawPow     |
| ERG  | erg.kryptex.network:3333  | Autolykos2 |
| ALPH | alph.kryptex.network:3333 | SHA3x      |

### B. MoneroOcean & Unmineable URLs

| POOL              | URL                           | PROTOCOL    |
|-------------------|-------------------------------|-------------|
| MoneroOcean (TCP) | gulf.moneroocean.stream:10128 | stratum+tcp |
| MoneroOcean (SSL) | gulf.moneroocean.stream:20128 | stratum+ssl |
| unMineable (TCP)  | rx.unmineable.com:3333        | stratum+tcp |
| unMineable (SSL)  | rx.unmineable.com:443         | stratum+ssl |
| Binance Pool      | bs.poolbinance.com:1800       | stratum+tcp |

### C. BIP44 Derivation Paths

| COIN           | PATH             | ADDRESS PREFIX                  |
|----------------|------------------|---------------------------------|
| Bitcoin (BTC)  | m/44'/0'/0'/0/0  | 1... (P2PKH) / bc1... (P2WPKH)  |
| Litecoin (LTC) | m/44'/2'/0'/0/0  | L... (P2PKH) / ltc1... (P2WPKH) |
| Dash (DASH)    | m/44'/5'/0'/0/0  | X... (P2PKH)                    |
| Ethereum (ETH) | m/44'/60'/0'/0/0 | 0x... (EIP-55 checksum)         |
| Monero (XMR)   | Monero specific  | 4... or 8... (base58 Monero)    |

### D. Troubleshooting Quick Reference

| SYMPTOM               | CAUSE                         | FIX                                                                 |
|-----------------------|-------------------------------|---------------------------------------------------------------------|
| Dashboard not loading | App not running / port in use | Run ./status.sh or python3 universal_miner.py to see startup errors |

| SYMPTOM                       | CAUSE                                    | FIX                                                                            |
|-------------------------------|------------------------------------------|--------------------------------------------------------------------------------|
| XMRig not found               | Installer did not complete build         | ls bin/xmrig - if missing, re-run installer. Check build.log for cmake errors. |
| Pool URL empty error          | Profile missing pool URL                 | Setup -> Pool tab -> click AUTO-FILL, or enter manually                        |
| Build failed armv7l           | Low RAM or missing build tools           | Need 1GB+ free RAM, 500MB+ disk. apt install libuv1-dev libssl-dev             |
| Port 8080 in use              | Another process on 8080                  | Edit universal_miner.py: change port=8080 to port=5002                         |
| 0.00 H/s hashrate             | Miner started but not connecting to pool | Check pool URL, credentials. View log: /api/log?lines=100                      |
| Swarm nodes all dead          | OOM on low-RAM device                    | Reduce TOTAL_NODES in miner_config.py or reduce NODE_THREADS                   |
| Wallet balance shows 0        | Balance API rate limited or unreachable  | Wait 5 min and refresh. Check internet connection.                             |
| Profit switcher not switching | Gain below MIN_SWITCH_GAIN_PCT threshold | Lower threshold via /api/profit/set_threshold or wait for market movement      |

### E. File Size Reference

| FILE                       | SIZE    | CONTENT TYPE                              |
|----------------------------|---------|-------------------------------------------|
| wallet_core.py             | ~43 KB  | BIP39/32/44 implementation, TX signing    |
| miner_config.py            | ~41 KB  | Coin/pool/algo registry, DB, profile CRUD |
| miner_swarm.py             | ~26 KB  | SwarmEngine, 40 SwarmNode orchestration   |
| templates/wallet_ui.html   | ~39 KB  | Full wallet UI with all panels            |
| templates/setup.html       | ~39 KB  | Setup wizard with dynamic forms           |
| templates/dashboard.html   | ~34 KB  | Live dashboard with SSE + charts          |
| wallet_validator.py        | ~34 KB  | Pre-flight config validation              |
| ump_miner_registry.py      | ~29 KB  | Extended coin/algo/miner registry         |
| universal_miner.py         | ~36 KB  | Flask app core + all routes               |
| ump_profit_switcher.py     | ~23 KB  | Profit switching daemon                   |
| ump_hardware_detect.py     | ~24 KB  | Hardware capability detection             |
| install_universal_miner.sh | ~16 KB  | Bash installer + XMRig source build       |
| TOTAL (all files)          | ~526 KB | Complete deployable package               |